

**FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA**  
**ANÁLISE E DESENVOLVIMENTO DE SISTEMAS**

GUSTAVO SOUZA NASCIMENTO  
ANDRÉ LUCATS  
RENAN LIMA SANTOS

**NEXORATECH**

CHATBOT DE APOIO À EQUIPE DA TURMA DO BEM

SÃO PAULO  
2026

## SUMÁRIO

|                                                  |   |
|--------------------------------------------------|---|
| 1. OBJETIVO E ESCOPO DO PROJETO.....             | 3 |
| 2. PRINCIPAIS FUNCIONALIDADES IMPLEMENTADAS..... | 3 |
| 3. TABELA DE ENDPOINTS.....                      | 5 |
| 4. PROTÓTIPO DO SISTEMA.....                     | 5 |
| 5. MODELO DE ENTIDADE-RELACIONAMENTO (MER).....  | 6 |
| 6. DIAGRAMA DE CLASSES.....                      | 6 |
| 7. PROCEDIMENTO PARA RODAR A APLICAÇÃO.....      | 7 |

## 1. OBJETIVO E ESCOPO DO PROJETO

A ONG Turma do Bem tem como missão conectar dentistas voluntários a pacientes em situação de vulnerabilidade, garantindo acesso gratuito a tratamentos odontológicos. O presente sistema foi desenvolvido em Java como solução de BackEnd para centralizar o cadastro e gerenciamento das informações operacionais da ONG. A aplicação permite o cadastro, consulta, atualização e remoção de seis entidades principais: Usuário Admin, Voluntário, Paciente, Triagem, Tratamento e Agendamento.

Nesta Sprint 4, o sistema foi evoluído para uma API RESTful feita com Quarkus, sendo utilizada no front-end React. Foram adicionadas camadas de BO (Business Object) com validações de regras de negócio e exceções customizadas.

## 2. PRINCIPAIS FUNCIONALIDADES IMPLEMENTADAS

- **CRUD Completo (6 entidades)**

Cadastro, listagem, busca por ID, atualização e remoção de Usuário, Voluntário, Paciente, Triagem, Tratamento e Agendamento.

- **API RESTful com Quarkus**

Endpoints (GET, POST, PUT, DELETE) para cada entidade.

- **Camada BO com Regras de Negócio**

Cada entidade possui um BO que valida dados antes de serem inseridos no banco de dados.

- **Exceções Customizadas**

*RegraDeNegocioException*, *RegistroDuplicadoException*, *EntidadeNaoEncontradaException*, *OperacaoNaoPermitidaException* e *ExcecoesConexao* garantem respostas claras ao front-end.

### 2.1 Paciente

**Inserir:** Percorre todos os pacientes já cadastrados e compara o e-mail informado com cada registro existente. Impedindo cadastros repetidos pelo mesmo endereço de e-mail.

**Atualizar:** Consulta o banco pelo ID antes de atualizar. Se o retorno for null, o sistema não permite a requisição garantindo que só registros existentes sejam modificados.

**Deletar:** Verifica se o paciente existe. Em seguida, varre toda a tabela de agendamentos; se qualquer agendamento desse paciente estiver com status AGENDADO, o sistema não permite a requisição bloqueando a exclusão até que o agendamento seja cancelado ou concluído.

## 2.2 Voluntário

**Inserir:** Analiza todos os voluntários cadastrados e verifica dois campos simultaneamente: e-mail e CRO. Qualquer duplicidade em um desses dois campos resulta em `RegistroDuplicadoException`, garantindo unicidade tanto do contato quanto do registro profissional do dentista/voluntário.

**Atualizar:** Não possui validação adicional, delega diretamente a atualização, o que significa que o front-end deve garantir os dados antes de chamar este endpoint.

**Deletar:** Busca o voluntário pelo ID; se não existir, lança `EntidadeNaoEncontradaException`. Não há bloqueio por vínculos ativos, diferente do paciente, o voluntário pode ser removido mesmo com tratamentos ou agendamentos associados.

## 2.3 Usuário Administrativo

**Inserir:** Aplica duas validações em sequência: primeiro verifica se o perfil informado pertence ao conjunto permitido (Admin, Leitura, Operador), lançando `RegraDeNegocioException` se inválido; depois percorre os usuários existentes e rejeita e-mail duplicado com `RegistroDuplicadoException`.

**Atualizar:** Reaplica a validação de perfil antes de persistir a atualização. Isso impede que uma edição introduza um perfil inválido no sistema, mesmo que o registro já exista.

**Deletar:** Confirma existência do usuário pelo ID antes de excluir. Se o ID não for encontrado, lança `EntidadeNaoEncontradaException`.

## 2.4 Triagem

**Inserir:** Valida a urgência contra a lista Baixa, Media, Alta. Depois percorre todas as triagens existentes: se o mesmo paciente já possuir uma triagem com status Concluida, lança `RegraDeNegocioException` exigindo que ela seja encerrada antes de abrir uma nova.

**Atualizar:** Valida o status informado contra o conjunto Concluida, Pendente, Cancelada antes de persistir. Qualquer valor fora desse conjunto resulta em `RegraDeNegocioException`.

**Deletar:** Confirma existência da triagem. Após isso, verifica se o status atual é exatamente Cancelada; qualquer outro status bloqueia a exclusão com `OperacaoNaoPermitidaException`, forçando que triagens sejam canceladas antes de serem removidas.

## 2.5 Tratamento

**Inserir:** Valida o status contra os valores Em Andamento, Concluído, Cancelado. Se o status for null ou fora desse conjunto, lança `RegraDeNegocioException` antes de qualquer acesso ao banco.

**Atualizar:** Replica a mesma validação de status no momento da atualização, impedindo que um tratamento seja atualizado com um estado inválido.

**Deletar:** Busca o tratamento pelo ID (lança `EntidadeNaoEncontradaException` se não existir) e então verifica o status, se for Em Andamento, lança `OperacaoNaoPermitidaException`, exigindo que o tratamento seja cancelado ou concluído antes de poder ser excluído.

## 2.6 Agendamento

**Inserir:** Valida o status (Agendado, Realizado, Cancelado) e, em seguida, percorre todos os agendamentos existentes. Se o mesmo voluntário já tiver um agendamento com status Agendado no mesmo `dataHora`, lança `RegraDeNegocioException` por conflito de horário, protegendo a agenda do profissional voluntário.

**Atualizar:** Verifica existência do agendamento pelo ID. Se o status atual for Realizado, bloqueia qualquer alteração com `RegraDeNegocioException`, pois um agendamento já realizado é imutável. Depois revalida o novo status informado.

**Deletar:** Confirma existência e verifica o status atual: se for AGENDADO (comparação exata em maiúsculas), lança `OperacaoNaoPermitidaException`. O agendamento precisa ser cancelado ou realizado antes de ser excluído.

## 3. TABELA DE ENDPOINTS

| Tt URI's          | Verbo HTTP | Tt Status esperado |
|-------------------|------------|--------------------|
| /agendamento/{id} | DELETE     | 200 OK             |
| /agendamento      | POST       | 201 Created        |
| /agendamento      | GET        | 200 OK             |
| /agendamento/{id} | GET        | 200 OK             |
| /agendamento      | PUT        | 200 OK             |
| /usuario/{id}     | DELETE     | 200 OK             |
| /usuario          | POST       | 201 Created        |

|                  |          |             |
|------------------|----------|-------------|
| /usuario         | GET ▾    | 200 OK      |
| /usuario/{id}    | GET ▾    | 200 OK      |
| /usuario         | PUT ▾    | 200 OK      |
| /paciente/{id}   | DELETE ▾ | 200 OK      |
| /paciente        | POST ▾   | 201 Created |
| /paciente        | GET ▾    | 200 OK      |
| /paciente/{id}   | GET ▾    | 200 OK      |
| /paciente        | PUT ▾    | 200 OK      |
| /tratamento/{id} | DELETE ▾ | 200 OK      |
| /tratamento      | POST ▾   | 201 Created |
| /tratamento      | GET ▾    | 200 OK      |
| /tratamento/{id} | GET ▾    | 200 OK      |
| /tratamento      | PUT ▾    | 200 OK      |
| /triagem/{id}    | DELETE ▾ | 200 OK      |
| /triagem         | POST ▾   | 201 Created |
| /triagem         | GET ▾    | 200 OK      |
| /triagem/{id}    | GET ▾    | 200 OK      |
| /triagem         | PUT ▾    | 200 OK      |
| /voluntario/{id} | DELETE ▾ | 200 OK      |
| /voluntario      | POST ▾   | 201 Created |
| /voluntario      | GET ▾    | 200 OK      |
| /voluntario/{id} | GET ▾    | 200 OK      |
| /voluntario      | PUT ▾    | 200 OK      |

## 4. PROTÓTIPO DO SISTEMA

### 4.1 VISUALIZAÇÃO DE DADOS

Pagina dedicada a visualizar as informações de Pacientes, Voluntários, Agendamentos, e triagens.

#### Visualização de Pacientes

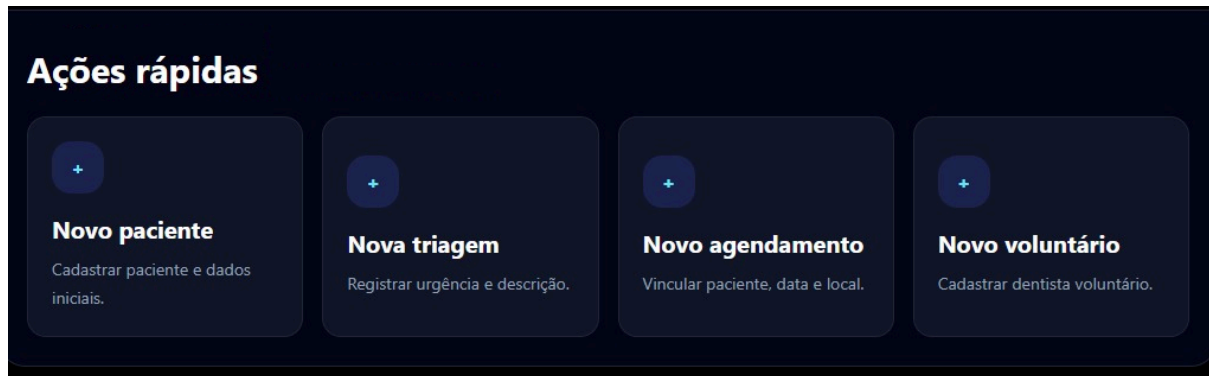
| Nome             | CPF         | Telefone    | Status       | Urgência | Ações        |
|------------------|-------------|-------------|--------------|----------|--------------|
| Richard          | 55564577788 | 11977660005 | Ativo        | Baixa    | Ver detalhes |
| Lucas Oliveira   | 11122233344 | 11977770001 | Ativo        | Baixa    | Ver detalhes |
| Maria Santos     | 22233344455 | 11977770002 | Em_Andamento | Alta     | Ver detalhes |
| José Fernandes   | 33344455566 | 11977770003 | Ativo        | Baixa    | Ver detalhes |
| Beatriz Carvalho | 44455566677 | 11977770004 | Inativo      | Alta     | Ver detalhes |

#### Visualização de Voluntários

| Nome               | CRO        | Telefone    | E-mail                   |
|--------------------|------------|-------------|--------------------------|
| Dr. Sergio Ramos   | CRO-SP-120 | 11988456005 | sergio.ramos@odonto.com  |
| Dra. Ana Paula     | CRO-SP-001 | 11988880001 | ana.paula@odonto.com     |
| Dr. Bruno Ferraz   | CRO-SP-002 | 11988880002 | bruno.ferraz@odonto.com  |
| Dra. Camila Torres | CRO-SP-003 | 11988880003 | camila.torres@odonto.com |
| Dr. Diego Martins  | CRO-SP-004 | 11988880004 | diego.martins@odonto.com |
| Dra. Elisa Ramos   | CRO-SP-005 | 11988880005 | elisa.ramos@odonto.com   |

## 4.2 CADASTRO DE DADOS

Local onde é possível adicionar, editar informações de Pacientes, Voluntários, Triagens e Agendamentos.



### Cadastro de Pacientes

CADASTRO

Novo paciente

Preencha os dados principais do paciente.

Nome

nome

Cpf

cpf

Telefone

telefone

Email

email

Cancelar

Salvar cadastro



## Cadastro de Triagem

CADASTRO

×

### Nova triagem

Registre a descrição, urgência e status da triagem.

|                                       |                                        |
|---------------------------------------|----------------------------------------|
| Paciente                              | Descricao                              |
| <input type="text" value="paciente"/> | <input type="text" value="descricao"/> |
| Urgencia                              | Status                                 |
| <input type="text" value="urgencia"/> | <input type="text" value="status"/>    |

## Cadastro de Agendamento

CADASTRO

×

### Novo agendamento

Informe paciente, voluntário, data e local.

|                                       |                                         |
|---------------------------------------|-----------------------------------------|
| Paciente                              | Voluntario                              |
| <input type="text" value="paciente"/> | <input type="text" value="voluntario"/> |
| DataHora                              | Local                                   |
| <input type="text" value="dataHora"/> | <input type="text" value="local"/>      |
| Status                                |                                         |
| <input type="text" value="status"/>   |                                         |

## Cadastro de Voluntário

CADASTRO

×

# Novo voluntário

Cadastre os dados do dentista voluntário.

Nome

nome

Cro

cro

Telefone

telefone

Email

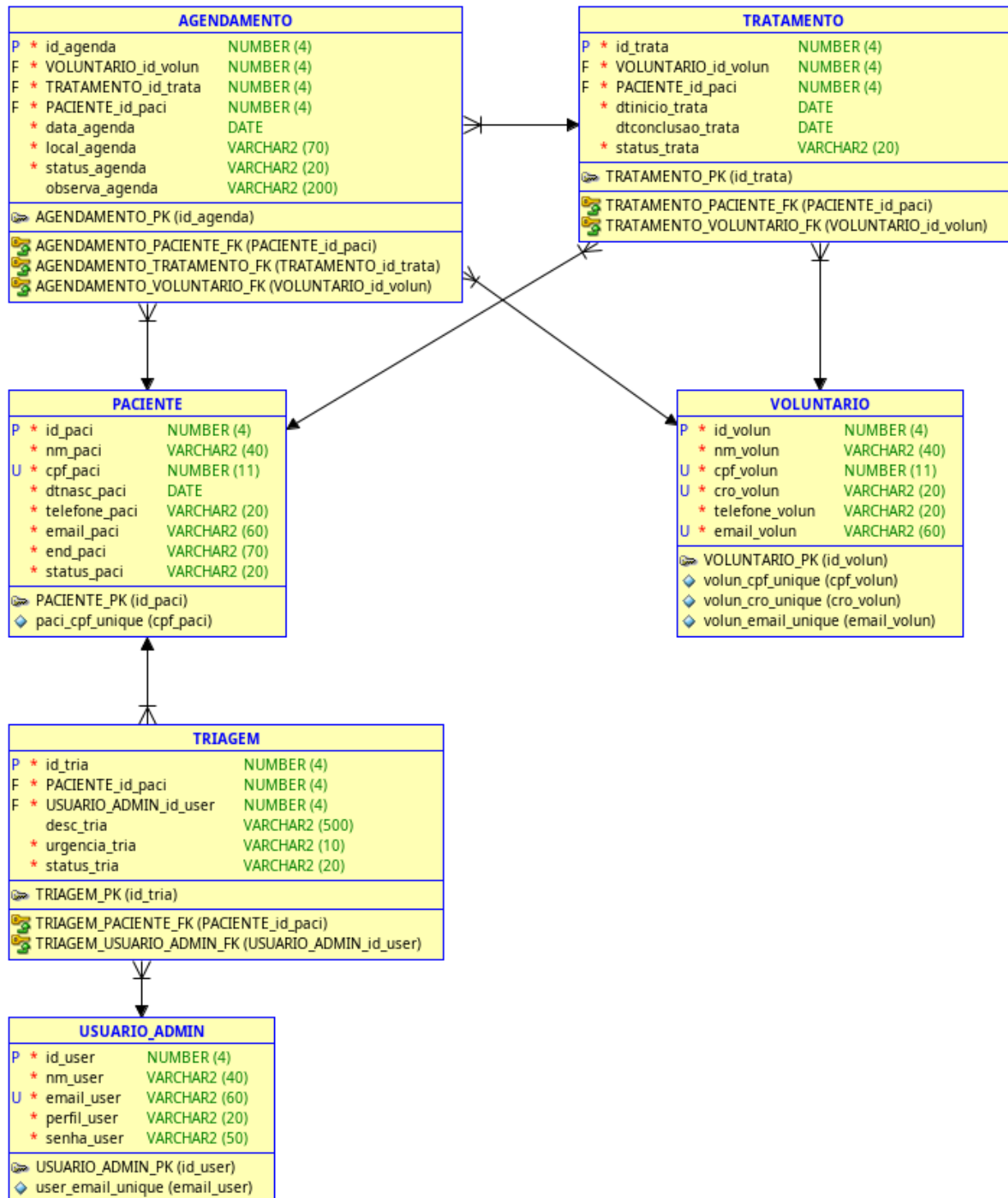
email

Cancelar

Salvar cadastro

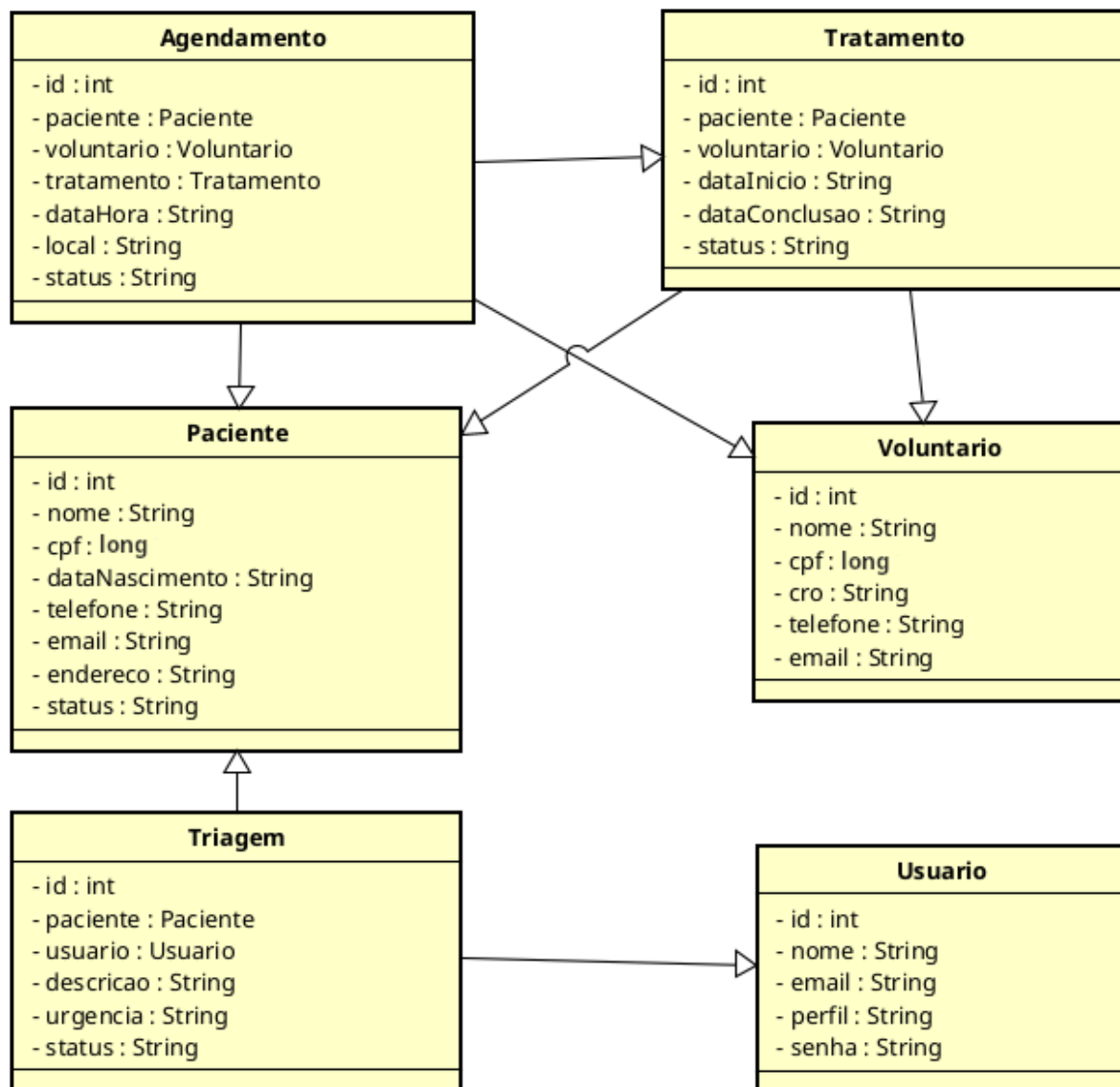
## 5. MODELO DE ENTIDADE-RELACIONAMENTO (MER)

O diagrama abaixo representa as 6 entidades do sistema e seus relacionamentos. Garantindo validação imediata.



## 6. DIAGRAMA DE CLASSE

O diagrama de classes representa a estrutura do BackEnd Java implementado. As 6 classes de entidade correspondem às tabelas Oracle, encapsulando atributos, construtores completo e vazio, getters/setters e toString(). Os DAOs implementam a camada de persistência via JDBC. Os BOs contêm as regras de negócio. Os Resources expõem os endpoints da API.



## 7. PROCEDIMENTO PARA RODAR A APLICAÇÃO

A API está publicada e disponível publicamente via Render.com. Não é necessário instalar nenhuma dependência para consumir os endpoints. Basta utilizar um cliente HTTP como Postman, Insomnia ou o próprio navegador para requisições GET.

### 7.1 URL DA API

<https://nexoratech-restfullapi.onrender.com/>

### 7.2 RODAR A APLICAÇÃO LOCALMENTE

Também é possível rodar a aplicação localmente a instalando via GitHub, clone o repositório do GitHub e abra a pasta nexoratech-restfullAPI/ no IntelliJ via File > Open. O IntelliJ detectará automaticamente o pom.xml e baixará as dependências.

Para executar, basta ter o Quarks-tools e executar o projeto normalmente ou via terminal utilizando esse comando: `./mvnw quarkus:dev`

A API subirá em `http://localhost:8080`. Teste o endpoint `/hello` para confirmar: deve retornar "Hello from Quarkus REST".